

- Description

This module manages a medical diagnosis and patient priority system, ensuring that:

1. **Patients are assigned a severity score (0-100) based on symptoms and medical evaluation.**
2. **Patients with higher severity receive priority for treatment.**
3. **If two patients have the same severity, priority is given based on arrival time.**
4. **A log of all diagnoses and treatments** is maintained for tracking and analytics.

This system ensures that critically ill patients receive timely treatment while maintaining fairness in patient triage.

- Specification

module MedicalDiagnosisSystem

var patients: sequence of (string, integer, real)

var treatmentLog: sequence of string

process admitPatient(name: string, severityScore: integer) flag: Boolean

functional scenario form:

severityScore $\geq$ 0 and severityScore $\leq$ 100 $\wedge$ flag = true and patients = patients $\oplus$ (name, severityScore, currentTime) and treatmentLog = treatmentLog $\oplus$ ("Admitted: " + name + " - Severity " + severityScore), Valid patient, admitted successfully
||

severityScore < 0 || severityScore > 100 $\wedge$ flag = false, Invalid severity score, admission rejected

end process

process treatNextPatient()

functional scenario form:

(|patients| > 0) $\wedge$ patients = removeHighestPriorityPatient(patients)
 $\wedge$ treatmentLog = treatmentLog $\oplus$ "Patient treated", Patient treated successfully

OR

(|patients| = 0) $\wedge$ No patients waiting, no action taken

end process

end module

- Java Code

```
package code;
import java.util.*;
class Patient {
    private String name;
    private int severityScore;
    private long arrivalTime;
    public Patient(String name, int severityScore) {
        this.name = name;
```

```

        this.severityScore = severityScore;
        this.arrivalTime = System.currentTimeMillis();
    }
    public String getName() {
        return name;
    }
    public int getSeverityScore() {
        return severityScore;
    }
    public long getArrivalTime() {
        return arrivalTime;
    }
}
class Hospital {
    private PriorityQueue<Patient> patientQueue;
    private List<String> treatmentLog;
    public Hospital() {
        this.treatmentLog = new ArrayList<>();
        this.patientQueue = new PriorityQueue<>((p1, p2) -> {
            if (p2.getSeverityScore() != p1.getSeverityScore()) {
                return Integer.compare(p2.getSeverityScore(), p1.getSeverityScore());
            } else {
                return Long.compare(p1.getArrivalTime(), p2.getArrivalTime());
            }
        });
    }
    public boolean admitPatient(String name, int severityScore) {
        if (severityScore >= 0 && severityScore <= 100) {
            Patient newPatient = new Patient(name, severityScore);
            patientQueue.add(newPatient);
            treatmentLog.add(String.format("Admitted: %s - Severity %d", name, severityScore));
            System.out.printf("Patient admitted: %s - Severity: %d\n", name, severityScore);
            return true;
        } else {
            System.out.println("Invalid severity score! Must be between 0 and 100.");
            return false;
        }
    }
    public void treatNextPatient() {

```

```

        if (!patientQueue.isEmpty()) {
            Patient next = patientQueue.poll();
            System.out.printf("Treating patient: %s - Severity: %d\n",
next.getName(), next.getSeverityScore());
            treatmentLog.add(String.format("Treated: %s - Severity %d",
next.getName(), next.getSeverityScore()));
        } else {
            System.out.println("No patients waiting for treatment.");
        }
    }
    public void printTreatmentLog() {
        System.out.println("\nTreatment History:");
        for (String log : treatmentLog) {
            System.out.println(log);
        }
    }
}

public class c11_MedicalDiagnosisSystem {
    public static void main(String[] args) throws InterruptedException
    {
        Hospital hospital = new Hospital();
        // testcase-VT:
        //     hospital.admitPatient("Alice", 2);
        //     Thread.sleep(10);
        //     hospital.admitPatient("Bob", 93);
        //     Thread.sleep(10);
        //     hospital.admitPatient("Charlie", -4);
        //     Thread.sleep(10);
        //     hospital.admitPatient("David", 93);
        //     Thread.sleep(10);
        //     hospital.admitPatient("Eve", 101);
        //     Thread.sleep(10);
        //     hospital.treatNextPatient();
        //     hospital.treatNextPatient();
        //     hospital.treatNextPatient();
        //     hospital.treatNextPatient();
        //     hospital.treatNextPatient();
        // testcase-FT:
        //     hospital.admitPatient("Ye-S", 1701211753);
        //     Thread.sleep(10);
        //     hospital.admitPatient("ezAo 幽", 0);
        //     Thread.sleep(10);
        //     hospital.admitPatient("???@", 0);
        //     Thread.sleep(10);
    }
}

```

```
//      hospital.admitPatient("", 0);
//      Thread.sleep(10);
//      hospital.admitPatient(" ??", 1701211699);
//      Thread.sleep(10);
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
// testcase-Z3:
//      hospital.admitPatient("Alice", 4);
//      Thread.sleep(10);
//      hospital.admitPatient("Bob", -1);
//      Thread.sleep(10);
//      hospital.admitPatient("Charlie", -1);
//      Thread.sleep(10);
//      hospital.admitPatient("David", 101);
//      Thread.sleep(10);
//      hospital.admitPatient("Eve", 101);
//      Thread.sleep(10);
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();
//      hospital.treatNextPatient();

// testcase-UVT:
hospital.admitPatient("Alice", 2);
Thread.sleep(10);
hospital.admitPatient("Bob", 93);
Thread.sleep(10);
hospital.admitPatient("Charlie", -4);
Thread.sleep(10);
hospital.admitPatient("David", 90);
Thread.sleep(10);
hospital.admitPatient("Eve", 101);
Thread.sleep(10);
hospital.treatNextPatient();
hospital.treatNextPatient();
hospital.treatNextPatient();
hospital.treatNextPatient();
hospital.treatNextPatient();

hospital.printTreatmentLog();
```

```

    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	818	828
✓ src	1.2 %	10	818	828
✓ code	1.2 %	10	818	828
✓ c11_MedicalDiagnosisSystem.java	100.0 %	10	0	10
> c11_MedicalDiagnosisSystem		0	0	0
> Hospital	100.0 %	10	0	10
> Patient		0	0	0

3. Vibration testing instruction coverage: 98.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.5 %	228	15,388	15,616
✓ src	1.5 %	228	15,388	15,616
✓ code	1.5 %	228	15,388	15,616
✓ c11_MedicalDiagnosisSystem.java	98.7 %	228	3	231
> c11_MedicalDiagnosisSystem	94.5 %	52	3	55
> Hospital	100.0 %	155	0	155
> Patient	100.0 %	21	0	21

4. Fuzz testing branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	824	832
✓ src	1.0 %	8	824	832
✓ code	1.0 %	8	824	832
✓ c11_MedicalDiagnosisSystem.java	80.0 %	8	2	10
> Hospital	80.0 %	8	2	10
> c11_MedicalDiagnosisSystem		0	0	0
> Patient		0	0	0

5. Fuzz testing instruction coverage: 96.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.4 %	222	15,444	15,666
✓ src	1.4 %	222	15,444	15,666
✓ code	1.4 %	222	15,444	15,666
✓ c11_MedicalDiagnosisSystem.java	96.1 %	222	9	231
> Hospital	96.1 %	149	6	155
> c11_MedicalDiagnosisSystem	94.5 %	52	3	55
> Patient	100.0 %	21	0	21

6. Z3 branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	832	840
✓ src	1.0 %	8	832	840
✓ code	1.0 %	8	832	840
✓ c11_MedicalDiagnosisSystem.java	80.0 %	8	2	10
> Hospital	80.0 %	8	2	10
> c11_MedicalDiagnosisSystem		0	0	0
> Patient		0	0	0

7. Z3 instruction coverage: 90%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.9 %	208	21,885	22,093
✓ src	0.9 %	208	21,885	22,093
✓ code	0.9 %	208	21,885	22,093
✓ c11_MedicalDiagnosisSystem.java	90.0 %	208	23	231
> Patient	85.7 %	18	3	21
> Hospital	89.0 %	138	17	155
> c11_MedicalDiagnosisSystem	94.5 %	52	3	55

8. Unimproved V-method branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.1 %	9	831	840
▼ src	1.1 %	9	831	840
▼ code	1.1 %	9	831	840
▼ c11_MedicalDiagnosisSystem.java	90.0 %	9	1	10
> Hospital	90.0 %	9	1	10
> c11_MedicalDiagnosisSystem		0	0	0
> Patient		0	0	0

9. Unimproved V-method instruction coverage: 94.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	219	21,874	22,093
▼ src	1.0 %	219	21,874	22,093
▼ code	1.0 %	219	21,874	22,093
▼ c11_MedicalDiagnosisSystem.java	94.8 %	219	12	231
> Patient	85.7 %	18	3	21
> c11_MedicalDiagnosisSystem	94.5 %	52	3	55
> Hospital	96.1 %	149	6	155